Dissertation Proposal: A Systematic Taxonomy of Correctness Failures in LLM-Generated GPU Kernels
Name: Ali Ansari
Word count: 2891

1 Background and Description of the Proposed Project

Graphics Processing Units (GPUs) have become essential hardware for modern machine learning and scientific computing. Their ability to execute thousands of operations in parallel makes them significantly faster than traditional CPUs for tasks such as matrix multiplication, convolution, and other data-intensive computations (Kirk and Hwu, 2012). To take full advantage of GPU hardware, developers write specialised programs called CUDA kernels using NVIDIA's CUDA programming framework. However, writing correct and efficient CUDA kernels is a difficult task that requires deep understanding of parallel programming, memory hierarchies, thread synchronisation, and hardware-specific constraints. The difficulty of this task is reflected in the complexity of hand-optimised implementations such as FlashAttention (Dao et al., 2022; Dao, 2024), which required significant expert effort to achieve efficient GPU utilisation.

In recent years, large language models (LLMs) have shown impressive ability in generating source code across many programming languages (Chen et al., 2021). This has led researchers to investigate whether these models can also generate CUDA kernels, which would lower the expertise barrier and accelerate GPU software development (Valero-Lara et al., 2023). Early results from recent benchmarking studies are promising but reveal a puzzling pattern: models that use chain-of-thought reasoning produce fewer compilation and runtime errors than standard models, yet all models struggle equally when it comes to producing functionally correct outputs (Ouyang et al., 2025).

This gap between execution success and functional correctness has been observed in multiple studies but has not yet been systematically explained. No published work has investigated the specific ways in which different types of models fail when generating CUDA code, or what root causes drive these failures. This project addresses that gap by developing a detailed failure taxonomy for LLM-generated CUDA kernels and analysing how reasoning and non-reasoning models differ in their failure patterns.

2 Aims and Objectives

The aim of this project is to develop a systematic taxonomy of correctness failure modes in CUDA GPU kernels generated by large language models, and to characterise how reasoning and non-reasoning models fail differently across these categories.

The primary objectives of the project are as follows:

1. Set up the KernelBench evaluation framework on available GPU hardware (CMP graphics laboratory PCs as the primary platform, and UEA's HALI HPC cluster if accessible) and verify that the diagnostic pipeline produces reliable results against PyTorch reference implementations (Paszke et al., 2019).

2. Generate CUDA kernels from at least two reasoning models (DeepSeek R1, o3) and two non-reasoning models (DeepSeek V3, GPT-4o) across KernelBench Level 2 and Level 3 tasks using a standardised one-shot prompt.

3. Build and apply an automated four-stage failure diagnosis pipeline using NVCC compilation, compute-sanitizer, output fuzzing, and precision stress testing to classify each generated kernel into one of four failure categories: syntactic, runtime, functional, or silent.

4. Analyse the distribution of failure types across reasoning and non-reasoning models, identifying whether specific failure categories are associated with specific model types or specific kernel tasks.

In addition to these primary objectives, the project may also explore two further directions depending on available time, data, and resources. The first would be to investigate root causes of failures within each category, for example distinguishing between incorrect thread indexing, missing synchronisation, and numerical precision errors. While this is a realistic goal, its scope depends heavily on the volume and variety of failures collected during experiments, and it may need to be narrowed or adjusted as the project develops. The second would be to examine reasoning traces from models like DeepSeek R1, looking for patterns in the chain-of-thought output that appear before specific failure types. This is a more exploratory direction that would only be pursued if the primary objectives are completed and sufficient data is available. Neither of these additional objectives is required for the project to make a meaningful contribution, but both could strengthen the findings if conditions allow.

The major tasks associated with this project include environment setup and hardware access, API integration for model querying, benchmark task selection and prompt design, pipeline development and validation, data collection across all model and task combinations, statistical analysis, and dissertation writing.

3 Critical Review of Related Literature

3.1 LLM Code Generation and the CUDA Challenge

Large language models have established strong baselines for generating standard software code. Early foundation models demonstrated significant capabilities in languages like Python and C++ (Chen et al., 2021). Subsequent models trained extensively on programming datasets, such as StarCoder (Li et al., 2023), Code Llama (Rozière et al., 2023), and DeepSeek-Coder (Guo et al., 2024), further improved syntactic reliability. However, generating code for graphics processing units using CUDA presents a steep learning curve. While early efforts explored integrating LLMs into High-Performance Computing (Ding et al., 2023), writing efficient CUDA kernels requires a deep understanding of hardware architecture, memory hierarchies, and thread synchronisation (Kirk and Hwu, 2012). Because LLMs generally predict sequential tokens, they struggle to model the complex execution environment of a GPU, which intermediate languages like Triton (Tillet et al., 2019) explicitly try to abstract.

3.2 Evaluating CUDA Generation: Benchmarks and Limitations

To evaluate these programming capabilities, researchers have developed specialised benchmarking frameworks. KernelBench (Ouyang et al., 2025) and CUDABench (Zhu et al., 2026) focus on measuring the compilation success and execution performance of generated kernels. Parallel efforts explore the baseline capabilities of models specifically tuned for GPU code (Chen et al., 2025). To improve evaluation stability, recent work has introduced agentic frameworks like robust-kbench (Lange et al., 2025) and multi-agent optimisation systems such as Astra (Wei et al., 2025). Despite their clear utility, these tools share a significant methodological limitation. They primarily function as automated evaluators that report overall pass rates. By treating ker-

nel generation mostly as a binary success or failure, these benchmarks do not critically categorise specific errors.

3.3 The Reasoning vs. Correctness Discrepancy

A critical finding emerging from these evaluations is a distinct split between execution success and functional accuracy. Benchmarking results reveal that reasoning models consistently produce significantly fewer syntactic and runtime errors than their non-reasoning counterparts (Ouyang et al., 2025; Zhu et al., 2026). However, this improvement in compilation does not translate to algorithmic accuracy. Both reasoning and non-reasoning models struggle equally with functional correctness. As Nichols et al. (2026) note, there is a pressing need for true performance alignment in high-performance code generation. Generated kernels frequently compile and execute without triggering system errors, yet they produce mathematically incorrect outputs. This discrepancy indicates that while modern models learn CUDA syntax, they lack a fundamental grasp of parallel algorithmic logic.

3.4 Chain-of-Thought in Parallel Contexts

The ability of reasoning models to reduce execution errors connects directly to their underlying architecture. Techniques like chain-of-thought prompting encourage models to break problems into smaller sequential steps (Wei et al., 2022). Furthermore, reinforcement learning enhances this verification process, allowing models to self-correct syntactic mistakes before finalising an output (Guo et al., 2025). Recent work on teaching LLMs to iteratively identify and fix errors in their own code (Chen et al., 2023) has shown that self-correction can improve compilation rates. Scaling inference compute with repeated sampling (Brown et al., 2024) can also surface syntactically valid code more often. Advanced optimisation research, such as the contrastive reinforcement learning proposed in CUDA-L1 (Li et al., 2025), attempts to bridge performance gaps automatically. However, sequential reasoning explains exactly why compilation improves while correctness lags. Syntax rules and variable type checking are inherently linear tasks. Chain-of-thought processes do not naturally map to the non-linear, spatial reasoning required to manage parallel threads and avoid race conditions. Consequently, reasoning models fix their syntax but remain unable to correct flawed parallel logic.

3.5 Gap Statement

In conclusion, while existing literature successfully measures the overall performance of LLMs in CUDA generation, a significant analytical void remains. No published work has systematically characterised the specific failure modes of generated GPU code, nor has any study adequately explained the persistent gap between reasoning capabilities and functional correctness in parallel computing contexts. This project directly addresses this critical gap. By building a comprehensive, four-category failure taxonomy encompassing syntactic, runtime, functional, and silent failures, and by comparing models such as DeepSeek R1 and V3, this research will isolate how different architectures fail. This categorisation will provide the crucial insights necessary to understand the reasoning mismatch and improve automated high-performance computing generation.

4 Methodologies and Methods

This study uses an empirical, benchmark-driven methodology to systematically characterise how reasoning and non-reasoning large language models fail when generating CUDA GPU kernels.

The experimental design is based on a controlled comparison between model pairs that share the same base architecture, which helps isolate the effect of reasoning capability on failure behaviour.

4.1 Model Selection

The primary comparison pair is DeepSeek R1 (Guo et al., 2025) (reasoning) and DeepSeek V3 (DeepSeek-AI, 2025) (non-reasoning). These two models are selected because they share the same base model, which provides the cleanest possible experimental contrast. Secondary model pairs, such as Claude with extended thinking versus Claude Sonnet and o3 versus GPT-4o, will be used to check whether the observed patterns hold across different model families. All models receive the same one-shot baseline prompt taken from KernelBench. This controls for prompt variation and ensures that any differences in output reflect model capability rather than prompt design.

4.2 Benchmark Tasks

Tasks are drawn from KernelBench Levels 2 and 3 (Ouyang et al., 2025). Level 2 tasks require fusing multiple PyTorch operators into a single CUDA kernel, for example combining matrix multiplication with bias addition and activation. Level 3 tasks involve implementing complete neural network architectures as custom CUDA kernels. These levels are chosen because they produce the highest failure rates in previous work, which maximises the number of failures available for taxonomic analysis. Level 1 tasks, which involve single operators, are excluded because models achieve high pass rates on them and therefore produce too few failures.

4.3 Failure Diagnosis Pipeline

Each generated kernel goes through a four-stage diagnostic pipeline, as illustrated in Figure 1. First, the kernel is compiled using NVCC. If compilation fails, the full error message is logged and the failure is categorised as a syntactic failure. Second, kernels that compile successfully are executed under NVIDIA's compute-sanitizer tool, which detects memory access violations, race conditions, and illegal memory operations. These are classified as runtime failures. Third, kernels that run without errors are compared against PyTorch (Paszke et al., 2019) reference implementations through output fuzzing: the same operation is executed on multiple input shapes and sizes, and outputs are compared element-wise within a configurable tolerance. Kernels that produce incorrect results are classified as functional failures. Fourth, a more detailed analysis identifies silent failures, which are kernels that pass basic correctness checks but show subtle numerical errors under varied precision conditions or edge-case inputs.

4.4 Root Cause Classification

For each detected failure, the root cause is recorded. For example, a functional failure might stem from incorrect thread indexing, a precision loss during float32-to-float16 conversion, or a missing synchronisation barrier. To support consistent classification at scale, an LLM-based classifier may be used to assign root causes to detected failures. In this approach, the failed kernel and its associated error output are sent to an LLM with a structured prompt asking it to select a root cause from a predefined set of categories. The reliability of this automated classifier would be validated against a manually labelled subset of failures to ensure its accuracy. This allows the analysis to go beyond failure frequency and examine the distribution of failure causes across model types.

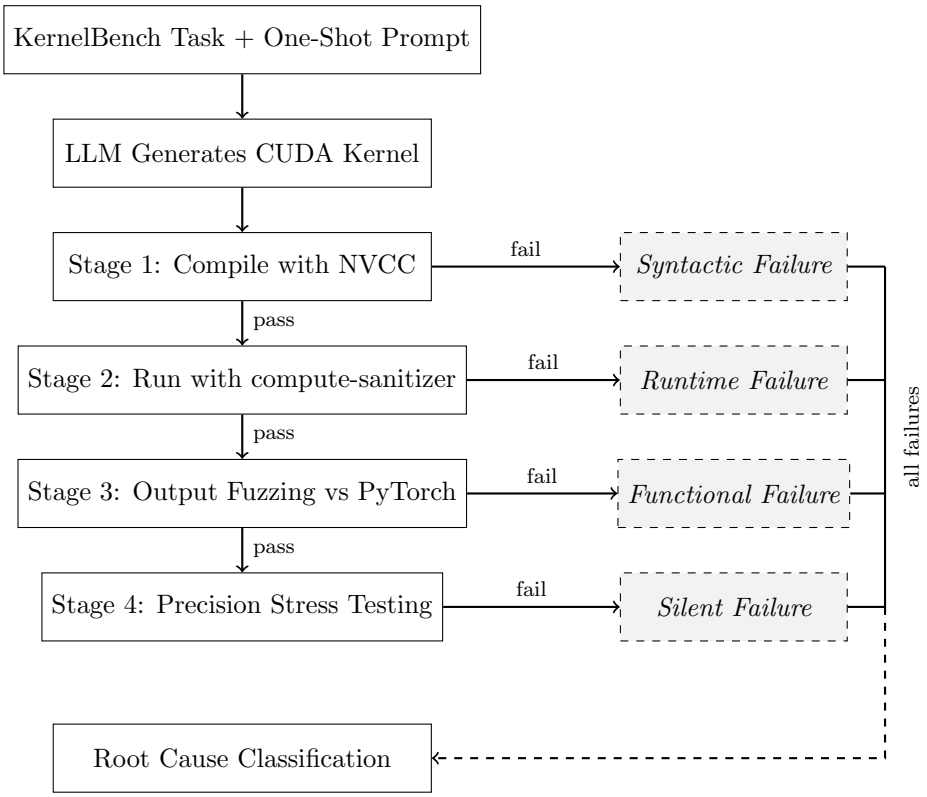


Figure 1: Four-stage failure diagnosis pipeline. Each generated kernel passes through the stages in order. If a kernel fails at any stage, it is classified accordingly and routed to root cause classification.

4.5 Hardware and Environment

Experiments will be conducted across two or three different GPU devices to strengthen the reliability of the results and add richness to the analysis. The primary platform is the graphics laboratory PCs in the CMP building, which are equipped with NVIDIA GPUs and are directly accessible. As a secondary option, UEA's HALI HPC cluster may also be used if access is available, providing NVIDIA RTX 6000 ADA GPUs with CUDA 12.1.1. Running the experiments on multiple devices allows the study to verify that the observed failure patterns are consistent across hardware configurations rather than being specific to a single setup. The conda environment uses Python 3.12.7 with PyTorch configured to use the available GPU as the reference device.

4.6 Analysis

The primary analysis cross-tabulates failure categories against model type (reasoning versus non-reasoning) and task level. Statistical tests will be used to assess whether reasoning models produce significantly different failure distributions. Secondary analyses will examine whether specific kernel types, such as GEMM, reductions, softmax, and layer normalisation, show distinct failure profiles. Where available, reasoning traces from models like DeepSeek R1 will also be examined to identify patterns that appear before specific failure types occur.

4.7 Evaluation of Success

The project will be considered successful if it produces a validated failure taxonomy with statistically meaningful differences in failure distributions between reasoning and non-reasoning

models, supported by diagnostic evidence that explains the observed patterns. The usefulness of the taxonomy is evaluated by whether it reveals actionable patterns. For instance, if reasoning models consistently avoid syntactic errors but introduce specific categories of silent failures, this would represent a novel and practically significant finding.

5 Analysis of Risks and Any Ethical Issues

5.1 Technical Risks

The most significant technical risk is related to API access and cost. Generating CUDA kernels from multiple commercial LLMs requires repeated API calls, and the total cost could become a constraint if the number of tasks and models is large. To manage this, the project prioritises DeepSeek R1 and V3 as the primary model pair, since their API costs are considerably lower through platform.deepseek.com compared to alternatives like OpenAI. Secondary model pairs will only be used if the budget allows. If API costs become prohibitive, the project can still produce meaningful results using only the primary pair, as these two models provide the cleanest comparison due to their shared base architecture.

A second risk is hardware availability. The primary platform for running experiments is the graphics laboratory PCs in the CMP building, which are equipped with NVIDIA GPUs and are directly accessible. As a secondary option, UEA's HALI HPC cluster may also be used if access can be arranged, which would provide additional compute capacity through its RTX 6000 ADA GPUs. Having two potential hardware options ensures that progress is not blocked if one platform becomes temporarily unavailable.

A third risk is that certain benchmark tasks may produce very few failures for some models, which would limit the statistical strength of the analysis. This is mitigated by focusing on KernelBench Level 2 and Level 3 tasks, which are known to produce high failure rates across all models tested in previous work (Ouyang et al., 2025).

5.2 Data and Licensing

The primary dataset used in this project is the KernelBench benchmark suite, which is publicly available on GitHub under the MIT licence. This licence permits free use, modification, and distribution for both academic and commercial purposes, so there are no restrictions on using this data for the project. The LLM-generated kernels produced during the experiments are original outputs and do not raise any intellectual property concerns.

5.3 Ethical Considerations

This project does not involve human participants, personal data, or sensitive information. All experiments are conducted using publicly available benchmark tasks and commercial LLM APIs under their standard terms of service. No ethical approval is required for this work.

6 Project Work Plan and Gantt Chart

The project spans approximately 16 weeks from mid-May to the end of August 2026. The work is organised into eight work packages, each with defined deliverables. The timeline is summarised in Figure 2.

WP1: Environment Setup and Hardware Access (Weeks 1–2). Install and configure the KernelBench framework, CUDA toolkit, PyTorch, and compute-sanitizer on the CMP graphics laboratory PCs. If HALI HPC access is granted, set up a mirrored environment there as well.

Validate the toolchain by running a small number of reference kernels. Deliverable: a working evaluation environment ready for experiments.

WP2: API Integration and Prompt Design (Weeks 2–3). Set up API access to DeepSeek R1, DeepSeek V3, and any secondary models. Prepare the standardised one-shot prompt based on the KernelBench format. Write scripts to automate kernel generation requests and store outputs in a structured format. Deliverable: automated kernel generation pipeline.

WP3: Diagnostic Pipeline Development (Weeks 3–5). Build the four-stage failure diagnosis pipeline: NVCC compilation, compute-sanitizer execution, output fuzzing against PyTorch references, and precision stress testing. Test each stage independently using manually crafted kernels with known failure types. Deliverable: validated diagnostic pipeline with logging and classification output.

WP4: Kernel Generation and Data Collection (Weeks 5–8). Run all selected KernelBench Level 2 and Level 3 tasks through the LLM APIs. Process each generated kernel through the diagnostic pipeline and record the failure category and associated error data. Begin with the primary model pair (DeepSeek R1 and V3) and extend to secondary pairs if time and budget allow. Deliverable: complete dataset of classified kernel failures.

WP5: Failure Classification and Root Cause Analysis (Weeks 7–10). Analyse the collected failures to identify root causes within each category. If the volume of failures is large, develop and validate the LLM-based root cause classifier against a manually labelled subset. Deliverable: annotated failure dataset with root cause labels.

WP6: Statistical Analysis and Results (Weeks 10–12). Perform cross-tabulation of failure categories against model type and task level. Apply statistical tests to assess whether differences in failure distributions are significant. Generate figures and tables summarising the key findings. Deliverable: complete set of results, figures, and statistical tests.

WP7: Dissertation Writing (Weeks 8–15). Begin writing the dissertation report in parallel with the later experimental work. Draft the introduction, literature review, and methodology chapters first, then add results and discussion as they become available. Incorporate feedback from the supervisor throughout. Deliverable: completed dissertation report.

WP8: Presentation Preparation and Submission (Weeks 15–16). Prepare the oral presentation slides summarising the project. Finalise and proofread the dissertation report. Submit the report and deliver the presentation. Deliverable: submitted dissertation and completed presentation.

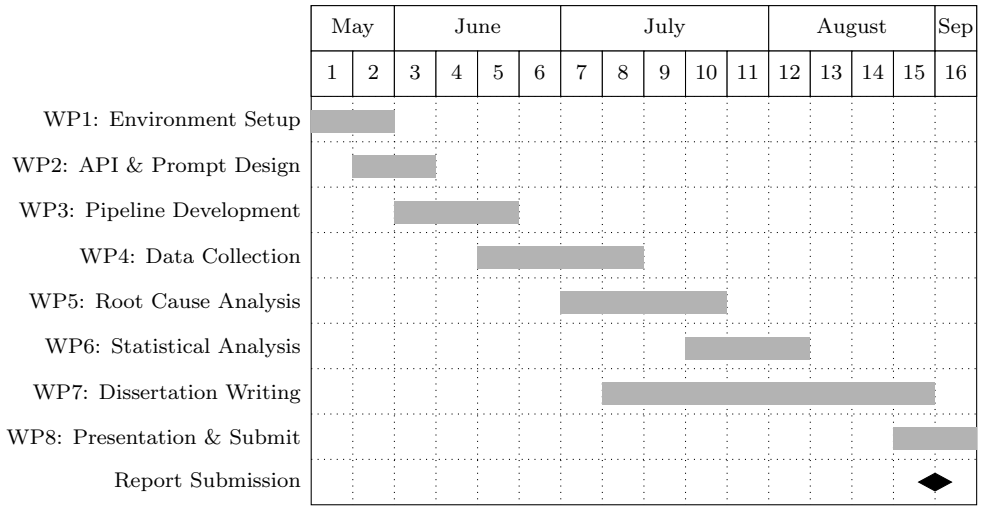


Figure 2: Gantt chart showing the project timeline across 16 weeks. Work packages are arranged to show dependencies, with overlapping periods where tasks run in parallel.

7 Conclusion

This proposal has outlined a project to systematically characterise how reasoning and non-reasoning large language models fail when generating CUDA GPU kernels. The project addresses a gap in the existing literature: while recent benchmarks have shown that reasoning models produce fewer compilation errors but no improvement in functional correctness, there is limited understanding of why this split occurs or what specific failure modes are involved. By building a four-category failure taxonomy and applying it across controlled model comparisons using KernelBench, this work aims to contribute a more detailed understanding of how LLM-generated GPU code breaks down. If successful, the findings could help inform future model development and practical strategies for using LLMs in high-performance computing workflows.

References

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. (2024). Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chen, W., Zhu, J., Fan, Q., Ma, Y., and Zou, A. (2025). CUDA-LLM: LLMs can write efficient CUDA kernels. *arXiv preprint arXiv:2506.09092*.

Chen, X., Lin, M., Schärli, N., and Zhou, D. (2023). Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*.

Dao, T. (2024). FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359.

DeepSeek-AI (2025). DeepSeek-V3 technical report.

Ding, X., Chen, L., Emani, M., Liao, C., Lin, P.-H., Vanderbruggen, T., Xie, Z., Cerpa, A., and Du, W. (2023). HPC-GPT: Integrating large language model for high-performance computing. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, SC-W '23, pages 951–960.

Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. (2024). DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.

Kirk, D. B. and Hwu, W.-m. W. (2012). *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2 edition.

Lange, R. T., Sun, Q., Prasad, A., Faldor, M., Tang, Y., and Ha, D. (2025). Towards robust agentic CUDA kernel benchmarking, verification, and optimization. *arXiv preprint arXiv:2509.14279*.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. (2023). StarCoder: may the source be with you! *Transactions on Machine Learning Research*.

Li, X., Sun, X., Wang, A., Li, J., and Shum, C. (2025). CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning. *arXiv preprint arXiv:2507.14111*.

Nichols, D., Polasam, P., Menon, H., Marathe, A., Gamblin, T., and Bhatele, A. (2026). Performance-aligned LLMs for generating fast HPC code. *IEEE Transactions on Parallel and Distributed Systems*.

Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Re, C., and Mirhoseini, A. (2025). KernelBench: Can LLMs write efficient GPU kernels? In *Forty-second International Conference on Machine Learning*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pages 10–19. Association for Computing Machinery.

Valero-Lara, P., Huante, A., Lail, M. A., Godoy, W. F., Teranishi, K., Balaprakash, P., and Vetter, J. S. (2023). Comparing LLaMA-2 and GPT-3 LLMs for HPC kernels generation. *arXiv preprint arXiv:2309.07103*.

Wei, A., Sun, T., Seenichamy, Y., Song, H., Ouyang, A., Mirhoseini, A., Wang, K., and Aiken, A. (2025). Astra: A multi-agent system for GPU kernel performance optimization. *arXiv preprint arXiv:2509.07506*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NeurIPS '22.

Zhu, J., Chen, W., Fan, Q., Ren, Z., Wu, J., Chai, X. Z., Rungrueangwutthinon, C., Ma, Y., and Zou, A. (2026). CUDAbench: Benchmarking LLMs for text-to-CUDA generation. *arXiv preprint arXiv:2603.02236*.